

SLM Guard — Technical Build Plan

v4 — Phase 1: Fraud Alert Triage Vertical

August 2026

This is a design and planning document only, except where noted below — the model-size decision in this revision is based on an actual on-device test, not a projection.

Revision note: v4 renames the product from SLM OS to SLM Guard. No architectural, technical, or roadmap content changed — this is a naming update only. All prior-version content (the 3B hardware-validated decision, distillation approach, evaluation harness, failure modes) is carried forward unchanged under the new name.

Guiding Constraint

Everything runs locally first, on a MacBook Air, with a Mac Mini arriving in roughly 45 days for heavier iteration. That rules out anything requiring a GPU cluster, external hosted training, or heavy MLOps infrastructure. The stack is Python end-to-end, minimal dependencies, built so pieces can be swapped for cloud-scale equivalents later without a redesign.

The MacBook Air in question is an **Apple M1 with 8GB unified memory** — tighter than the original "3B–7B" planning range assumed, which is why the model-size decision below was validated empirically rather than assumed.

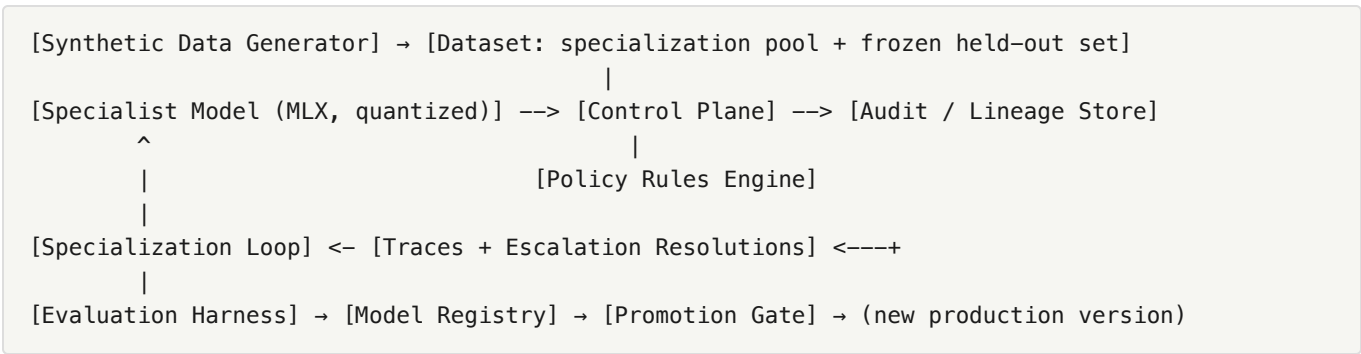
Phase 1 Hardware Validation (Completed)

Before committing to a model size, both ends of the originally planned range were run on-device via MLX, using the same fraud-triage-style prompt:

Model	Peak memory	Generation speed	Verdict
Qwen2.5-3B-Instruct-4bit	1.9 GB	~30 tokens/sec	Comfortable headroom on 8GB total RAM
Qwen2.5-7B-Instruct-4bit	4.4 GB	~10 tokens/sec	Runs for inference only, but already over half of total system RAM — before LoRA fine-tuning adds gradients, optimizer state, and activations on top

Decision: 3B is the committed model size for Phase 1. 7B is not abandoned — it's deferred to the Mac Mini stage of the roadmap (Phase 2), where more unified memory removes the constraint that makes it risky here. Building the whole Phase 1 loop around 3B also keeps the control plane, schema, and evaluation harness honest about what a genuinely resource-constrained specialist model can do, which is closer to the real deployment conditions this product argues for.

Component Architecture



Technology Choices

Layer	Choice	Rationale
Model runtime	MLX / mlx-lm	Native to Apple Silicon; supports both quantized inference and LoRA fine-tuning on-device — one framework covers inference and specialization without a cloud GPU
Model size	3B quantized, committed (Qwen2.5-3B-Instruct-4bit, validated on-device — see above). 7B deferred to the Mac Mini stage	Empirically validated, not assumed: 3B leaves comfortable headroom on 8GB; 7B does not
Control plane	Python + FastAPI	Simple recommend-endpoint now; can be extracted into a standalone service in Phase 2 without a rewrite
Output contract	JSON Schema + constrained/grammar decoding (e.g. outlines, or MLX grammar-constrained sampling)	The model is prevented from emitting an out-of-schema token in the first place. Treated as a Week 1 spike — see below
Policy rules	Declarative YAML rule set, evaluated by a small rule function	Avoids over-engineering with a rules engine like OPA/Rego at this stage; kept swappable behind a stable interface
Audit / lineage store	SQLite, append-only tables, each row hash-chained to the previous row	File-based, zero operational overhead; hash-chaining gives tamper-evidence sufficient for an MRM/audit conversation without needing a real blockchain
Synthetic data generation	A larger model (API-based) generates diverse fraud-alert scenarios and ground-truth recommendations, governed by an explicit quality rubric and rejection process — see below	Diversity and correctness must be verified before anything is trained on it — data quality is the single largest risk to the whole loop
Fine-tuning	mlx-lm LoRA, small rank, few epochs, adapters fused after training	Matches the "light QLoRA" approach in the roadmap; fusing keeps production inference

		simple — one quantized checkpoint, not adapter-swapping at serve time
Model registry	JSON manifest per version — parent version id, training-data hash, evaluation results, promotion status — stored alongside SQLite	Gives real lineage without adopting MLflow/W&B-class infrastructure this early
Orchestration	Plain Python CLI with subcommands (generate-data, run-baseline, specialize, evaluate, promote)	No workflow engine (Airflow/Prefect) at this scale — a CLI is auditable, debuggable, and matches the "narrow, not broad" build philosophy

Recommendation Schema (illustrative shape)

```
{
  "alert_id": "string",
  "action": "approve | decline | escalate_l2 | request_more_info",
  "risk_score": 0.0,
  "confidence": 0.0,
  "policy_flags": ["string"],
  "rationale": "string, length-bounded"
}
```

The control plane validates the schema, applies policy rules (e.g. certain transaction types always escalate regardless of model output; confidence below threshold forces escalation), makes the final decision, and writes the full trace — input, raw model output, validation result, policy checks applied, final decision, model version id — to the audit log.

Synthetic Data Quality Process

Flagged in review as the single highest point of failure: "human-reviewed sample" alone is necessary but not sufficient. Phase 1 uses three concrete artifacts instead:

1. **A written quality rubric** (companion 1-page document, produced before bulk generation begins). Defines what makes a synthetic fraud alert "good enough" to train on — realistic feature combinations, a correct and internally consistent ground-truth label, no contradictory or degenerate cases, rationale text that actually justifies the labeled action.
2. **A minimum diversity checklist**, covering at minimum: transaction types and channels; customer/account segments and risk profiles; edge cases (near-threshold amounts, first-time patterns, conflicting signals); policy-boundary cases (scenarios that sit exactly on a forced-escalation or hard-decline rule).
3. **An explicit rejection process, not just a review sample.** Every generated batch is scored against the rubric; batches falling below a defined pass rate are rejected and regenerated rather than folded in as-is. Spot-review of accepted batches still happens, but acceptance is gated by the rubric score, not by review alone.

Both the specialization pool and the frozen held-out set are drawn from rubric-passing data — a weak or biased dataset would silently invalidate both the baseline and every subsequent specialization cycle.

Specialization Loop

1. Pull traces since the last cycle from the audit store.
2. Filter to the traces that matter: low-confidence cases, cases where the policy engine overrode the model, and cases escalated to the larger model or a human.
3. Convert filtered traces into synthetic training examples that preserve the structural pattern of what made the case hard, without carrying the literal record — this is the privacy-safe conversion step, built for real now so it transfers directly to a deployment with genuinely sensitive data later. (See the product brief for the explicit statement of this step's current limits — it is careful rewriting plus review in Phase 1, not yet a formal privacy guarantee.)
4. Merge into the specialization pool and run a LoRA fine-tune; fuse adapters into a candidate model version.
5. Score the candidate against the frozen held-out set (never used in training) and the regression / challenge set (curated hard cases that must never break).
6. Apply the promotion gate (below).

Distillation Approach

Two distinct mechanisms are in use, and one is deliberately excluded:

In use — response-level (behavioral) distillation

- **Initial data generation:** the larger model produces the first synthetic scenario/label pairs used to establish the baseline specialist.
- **Continuous, via the escalation path:** every case the small model escalates (low confidence, policy override, or genuinely hard case) and that gets resolved by the larger model becomes a teacher demonstration. These resolutions feed directly into the next specialization cycle, making distillation continuous rather than a one-time step, and reusing the escalation mechanism that already exists for cost control.

In use — rationale distillation

The rationale field in the recommendation schema is trained to reflect the teacher's justification style, not just its final action — this improves interpretability and audit quality, not only accuracy.

Deliberately excluded — logit-level (soft-label) knowledge distillation

Classic KD trains the student to match the teacher's output probability distribution, not just its final answer. This is not used here because:

- An API-based teacher model does not expose token-level log-probabilities to distill against.
- Even with access, the teacher and student are unlikely to share a tokenizer or architecture family, making logit-matching meaningless without extra alignment work.

- Logit-level KD only becomes practical if the teacher is a larger open-weights model in the same family, run locally — more compute than Phase 1 warrants.

Net effect: the loop uses demonstration-level distillation (what the teacher decided and why), not distribution-level distillation (the exact shape of the teacher's confidence).

Evaluation Harness (fully specified)

Metrics

- Overall accuracy and per-class precision/recall on the 4-way action classification, with particular attention to the safety-critical classes (`decline`, `escalate_l2`) — a miss in either direction matters more than an average accuracy number captures.
- Policy violation rate: share of recommendations that would have violated a defined policy constraint had the control plane not intervened.
- Confidence calibration: measured via Expected Calibration Error (ECE) with a binned reliability diagram as the primary view; a simpler confidence-vs-correctness correlation is tracked as a secondary, easier-to-read signal.

Held-out test set

- Carved out once, before any training touches the data, and frozen for the life of Phase 1 — never regenerated or silently replaced.
- Stratified across all four action classes and explicitly including policy-boundary cases, not just easy/representative ones.
- Target size in line with the original data plan: roughly 150–300 examples out of the 800–1,500-example total dataset.

Regression / challenge set

Curated separately from the held-out set: hand-picked hard cases and any real failure found during development or evaluation. Only grows over time. A case is never silently dropped from this set once added — that would mask a known failure mode.

What "no regression" means, stated explicitly

Given Phase 1's held-out set size (150–300 examples), statistical significance testing would produce wide, not-very-actionable confidence intervals. Phase 1 therefore uses a fixed, explicit absolute-drop threshold (defined in configuration, not left implicit) rather than a significance test — e.g., accuracy on the frozen set must not fall by more than a stated number of percentage points versus the current production baseline. This threshold is a conscious, conservative choice for a small-sample Phase 1, and is expected to be replaced by a more statistically rigorous comparison once evaluation volumes grow in Phase 2.

Promotion Gate

Gate	Criterion
------	-----------

Accuracy	No regression against the frozen held-out set relative to the current production baseline, per the fixed threshold above
Policy / safety compliance	Zero tolerance on recommendations that violate defined policy constraints
Confidence calibration	Tracked (ECE + reliability diagram) from cycle one; becomes a hard gate only once a stable calibration baseline exists across cycles. Early cycles monitor it without blocking promotion on it
Regression / challenge set	No new failures on the curated set of edge cases and prior known failure patterns

No promotion is an explicit, valid outcome — not a failure. If a candidate doesn't clear every gate, or if a cycle simply hasn't produced enough new specialization signal to expect improvement, the loop records "no promotion this cycle" and leaves the production version unchanged.

Control Plane Failure Modes

Situation	Behavior
Model output is schema-valid but violates a policy rule	The policy engine overrides the model's recommendation. The override — and the fact that the model's raw output was schema-valid but rejected on policy grounds — is logged. The model is never trusted merely because its output parsed.
Confidence is high but the case falls in a forced-escalation category	The deterministic forced-escalation rule always wins, regardless of the model's confidence score. Confidence never overrides a policy-defined escalation requirement.
Model output fails schema validation	Hard failure. The case is automatically escalated (to the larger model or a human, per configuration) and logged as a schema failure — never retried silently or guessed at.
Model is unavailable or times out	Default fallback is automatic escalation, not a blocked or silently dropped case. The unavailability event itself is logged, since repeated timeouts are an operational signal worth tracking.

Constrained Decoding: Week 1 Spike

Constrained/grammar-based JSON generation is not a solved problem on MLX the way it is on the CUDA/vLLM/outlines side — tooling maturity there is real and shouldn't be assumed away. This is treated explicitly as a **Week 1 spike**, not a background assumption:

- First few days of Phase 1 are spent confirming whether reliable schema-constrained generation is achievable on MLX at 3B, and if not, what the closest workable alternative is (e.g., stricter prompt-level formatting plus aggressive post-generation validation).
- The fallback path is designed from day one regardless of spike outcome: **post-generation JSON Schema validation, with any failure automatically routed to the hard-failure/escalation path** already defined above. The system's correctness never depends on constrained decoding working perfectly — it depends on schema violations always being caught and escalated, one way or another.

Phase 1 Deliverable (5 weeks, MacBook Air)

A CLI-driven, end-to-end reproducible run producing: baseline accuracy → one specialization cycle (including at least one escalation-driven distillation pass) → post-cycle accuracy (or an honest "no promotion this cycle" result) → full audit trail export.

Known Technical Risks

- **Structured output reliability.** Addressed above via the Week 1 spike and the designed fallback path.
- **Synthetic data quality.** Addressed above via the rubric, diversity checklist, and rejection process — still the most consequential risk in the plan.
- **mlx-lm LoRA maturity.** Younger and less battle-tested than the CUDA/PEFT ecosystem. Worth a short spike early to confirm the fine-tuning and fusing workflow behaves as expected.
- **Escalation-driven distillation volume.** In early cycles, escalation volume may be too low or too narrow to meaningfully improve the model — this is exactly what the explicit "no promotion this cycle" outcome is designed to tolerate.
- **8GB memory ceiling during fine-tuning specifically.** Inference at 3B was validated at 1.9GB peak. LoRA fine-tuning adds gradients, optimizer state, and activation memory on top — this has not yet been validated on-device and is the next hardware check to run before committing to a training batch size.

Immediate Next Steps Before Implementation

1. Write the Synthetic Data Quality Rubric (1 page) — define "good enough to train on" and the batch-rejection threshold, before bulk data generation starts.
2. Run the constrained-decoding spike on MLX in the first 3–4 days; confirm the approach or fall back, and confirm the hard-failure → escalate path works either way.
3. Validate LoRA fine-tuning memory footprint at 3B on this machine specifically, before assuming a training batch size.
4. Define the exact evaluation metrics and the construction of the held-out and challenge sets before generating the bulk of the dataset — not after.
5. Confirm the "no promotion this cycle" outcome is implemented in the loop before the first specialization run.
6. Keep the control-plane decision-authority language sharp and consistent between the brief and the eventual implementation.

How This Maps to the Roadmap

Roadmap phase	This document covers
Phase 1 — Vertical reference implementation	Everything above: schema, control plane, baseline model (3B, validated), specialization loop, distillation approach, evaluation harness, failure modes

Phase 2 — Harden into platform core	Extract control plane and specialization engine into reusable, versioned components; move audit store to Postgres if needed; add a second domain pack to prove reusability; move toward formally verifiable privacy guarantees for the synthetic-conversion step; move toward statistically rigorous regression testing as evaluation volume grows; revisit 7B on the Mac Mini once its higher unified memory is available
Phase 3 — Platform shape & credibility	Open-source the core and reference implementation; publish architecture decision records
Phase 4 — Enterprise packaging	Defined separately once Phase 1 results exist

Status: design document, with one on-device validation completed (model-size decision above). Implementation has not started.